

Metals — metamodel pipeline

This notebook covers the metals — gold, silver, copper and platinum. Of the three asset classes this is the cleanest expression of the signal's contrarian construction: every metal reverses against its prior move. That tidiness at the signal level does not, however, translate into a strong metamodel edge. The most interesting tension in the whole study lives here — a feature set that looks predictive in cross-validation can still collapse to chance out of sample, and gold is the clearest example. The sections below follow that thread from the signal audit to the locked models and their honest hold-out scores.

Universe. `gc1s`, `si1s`, `hg1s`, `pl1s` (4 contracts).

Reading guide. The notebook runs top to bottom against committed artifacts; no model is retrained. Each stage loads its result files and embeds the figures already produced by the pipeline. Section order mirrors the marking scheme in `reports/harry/00-context.md`: data, signal, labelling, features, model selection, cluster importance, evaluation.

Source of truth. Where a number appears it is loaded from a file in this cell's output, not transcribed from prose; the few places where the committed data and the written reports disagree are flagged inline.

```
In [1]: CLASS = 'metals'
INSTS = ['gc1s', 'si1s', 'hg1s', 'pl1s']
NAMES = {'gc1s': 'Gold (GC)', 'si1s': 'Silver (SI)', 'hg1s': 'Copper (HG)',
        'pl1s': 'Platinum (PL)'}

from pathlib import Path
import pandas as pd
import warnings
warnings.filterwarnings("ignore")
from IPython.display import Image, display, Markdown

def _repo_root() -> Path:
    here = Path.cwd().resolve()
    for q in [here, *here.parents]:
        if (q / "pyproject.toml").exists():
            return q
    return here

REPO = _repo_root()
OUT = REPO / "src/stml/new_work/outputs"
RESULTS = REPO / "results"
REPORTS = REPO / "reports"
DATA = REPO / "data"
```

```

def show(path, width=1000):
    p = Path(path)
    if p.exists():
        display(Image(str(p), width=width))
    else:
        print(f"[missing figure] {p}")

def load(path) -> pd.DataFrame:
    p = Path(path)
    if p.exists():
        return pd.read_csv(p)
    print(f"[missing data] {p}")
    return pd.DataFrame()

def table(df, n=None, caption=None):
    if df is None or len(df) == 0:
        print("[empty table]")
        return
    d = df.head(n) if n else df
    sty = d.style.format(precision=4).hide(axis="index")
    if caption:
        sty = sty.set_caption(caption)
    display(sty)

```

1. Data and cleaning

The price history runs from 1990, while the primary signal is released only over 2020-01-03 to 2022-06-30; the sealed test period sits beyond the embargo. Loading goes through `stm1.io`, and the missing-data policy — keep zero-volume weekday settles, drop weekend and out-of-bounds rows — is documented in `reports/missing-data-report.md` and implemented in `stm1.na_checks`. The coverage table below is computed directly from `data/ohlcvc_data.csv` for this class's contracts.

```

In [2]: ohlcv = load(DATA / "ohlcvc_data.csv")
sub = ohlcv[ohlcv["instrument"].isin(INSTS)]
cov = (sub.groupby("instrument")
        .agg(first_date=("date", "min"), last_date=("date", "max"),
        n_rows=("date", "size"))
        .reindex(INSTS).reset_index())
table(cov, caption=f"{CLASS.title()} – OHLCV coverage
(data/ohlcvc_data.csv)")

```

Metals — OHLCV coverage (data/ohlc_data.csv)

instrument	first_date	last_date	n_rows
gc1s	1990-01-02	2022-06-30	8171
si1s	1990-01-02	2022-06-30	8172
hg1s	1990-01-02	2022-06-30	8172
pl1s	1990-01-02	2022-06-30	8169

2. Signal characterisation

Metals are the textbook case. All four contracts load negatively on the trailing one-day return and positively on the forward return, so the signal is uniformly short-horizon mean reversion with no awkward exceptions of the kind crude introduces in energy. If signal cleanliness alone decided the matter, metals would be the strongest class. The model section shows why it is not: a well-behaved signal is necessary but not sufficient for the metamodel to add value.

```
In [3]: sd = load(RESULTS / "harry/signal_direction.csv")
if not sd.empty:
    want = ["instrument", "n_bets", "corr_fwd_1", "corr_fwd_5",
"corr_trail_1", "corr_contemp_0"]
    cols = [c for c in want if c in sd.columns]
    rows = sd[sd["instrument"].isin(INSTS)]
    [cols].set_index("instrument").reindex(INSTS).reset_index()
    table(rows, caption=f"{CLASS.title()} – signal direction
(results/harry/signal_direction.csv)")
```

Metals — signal direction (results/harry/signal_direction.csv)

instrument	n_bets	corr_fwd_1	corr_fwd_5	corr_trail_1	corr_contemp_0
gc1s	168	0.1291	0.0152	-0.0870	-0.0153
si1s	578	0.0567	0.0018	-0.1434	-0.1577
hg1s	628	0.1218	0.0215	-0.1092	-0.1793
pl1s	557	0.0709	0.0243	-0.1475	-0.1870

3. Labelling

Each signal is turned into a binary meta-label by the triple-barrier method of López de Prado (2018, ch. 3): an upper (profit-taking) and lower (stop-loss) barrier are placed in units of a causal volatility estimate, a vertical barrier caps the holding period, and the label records whether taking the bet would have paid. Overlapping events are down-weighted by average uniqueness (López de Prado, 2018, ch. 4).

The submitted metamodel is trained and scored on the **team-provided labels** in `data/meta/triple_barrier_labels.csv`. The barrier geometry is tuned per instrument and held constant within each: the stop-loss sits at one quarter of the volatility estimate for every contract (`sl = 0.25`), while the profit-taking multiple and the vertical-barrier horizon are set per instrument — some contracts take a tight one-day barrier, others a wider profit target held for two to four weeks. The exact geometry is read from the data and shown per instrument below rather than quoted, because the values differ across the universe.

Provenance is not assumed: every locked out-of-sample prediction in `metamodel_predictions.csv` matches these labels exactly — 1373 of 1373 rows agree on the binary label after joining on instrument and date — so this file, not any other, is the label set the model was built on.

Provenance note. An earlier exploration documented in `reports/harry/02-labels.md` and persisted to `results/harry/events.csv` used a uniform ten-day barrier; it produces the same number of events (one per signal date) but different labels, and it was superseded by the per-instrument team labels used here. The report is retained as a record of that exploration.

```
In [4]: lab = load(DATA / "meta/triple_barrier_labels.csv")
lab = lab[lab["instrument"].isin(INSTS)]
if not lab.empty:
    # Per-instrument barrier geometry + event statistics, straight from the
    # team labels.
    g = (lab.groupby("instrument")
         .agg(pt=("pt", "first"), sl=("sl", "first"), h=("h", "first"),
              n_events=("label", "size"), label1_share=("label",
                                                         "mean"),
              mean_sigma=("sigma", "mean"))
         .reindex(INSTS).reset_index())
    table(g, caption=f"{CLASS.title()} - per-instrument barriers and labels
(data/meta/triple_barrier_labels.csv)")
    if "partition" in lab.columns:
        part = (lab.groupby(["instrument", "partition"]).size()
                 .unstack(fill_value=0).reindex(INSTS).reset_index())
        table(part, caption="Events per train / validation / test
partition")
    print(f"Total labelled events for {CLASS}: {len(lab):,} | label==1
share: {lab['label'].mean():.3f}")
```

Metals — per-instrument barriers and labels

(data/meta/triple_barrier_labels.csv)

instrument	pt	sl	h	n_events	label1_share	mean_sigma
gc1s	1.0000	0.2500	15	160	0.5000	0.0118
si1s	0.7500	0.2500	20	558	0.4140	0.0217
hg1s	1.0000	0.2500	10	618	0.4126	0.0148
pl1s	0.2500	0.2500	1	556	0.5396	0.0218

Events per train / validation / test
partition

instrument	test	train	val
gc1s	22	118	20
si1s	97	340	121
hg1s	115	377	126
pl1s	104	339	113

Total labelled events for metals: 1,892 | label==1 share: 0.458

4. Features

Predictors are grouped into causal families, all computed at the signal date and predictive of the next-day label, so none peeks into the future. The micro families (`reports/harry/03-features.md`) cover the signal's own trajectory, conditional risk, information-theoretic dependence, fixed microstructure proxies, cross-asset structure, an optional wavelet decomposition and a concept-drift discriminator. A macro block of thirty-one rolling, standardised series (`reports/harry/03-6-macro-features.md`) adds volatility, rates, credit, dollar, commodity-fundamental and growth context. Two hidden-Markov regime blocks — per-instrument volatility (`hmm_vol`) and a global risk-on/risk-off state (`hmm_macro`) — are fit only on pre-sample data and then frozen, so they are train-fitted rather than leaking. Features are tagged `E` (purely causal) or `TF` (train-fitted on pre-sample) accordingly.

```
In [5]: # The feature matrix is built by the pipeline; here we confirm the frozen
HMM
# regime features are present for this class's instruments.
for fn in ["features_hmm_vol.csv", "features_hmm_macro.csv"]:
    f = load(REPO / "src/stml/new_work" / fn)
    if not f.empty and "instrument" in f.columns:
        n = f[f["instrument"].isin(INSTS)].shape[0]
        print(f"{fn}: {n:,} rows for {CLASS} instruments | columns:
{list(f.columns)}")
    elif not f.empty:
        print(f"{fn}: {f.shape[0]:,} rows | columns: {list(f.columns)}")
```

```
features_hmm_vol.csv: 2,512 rows for metals instruments | columns: ['date',  
'instrument', 'hmm_vol_p0_calm', 'hmm_vol_p2_turbulent',  
'hmm_vol_next_turbulent', 'hmm_vol_entropy']  
features_hmm_macro.csv: 2,580 rows for metals instruments | columns:  
['date', 'instrument', 'hmm_macro_p0', 'hmm_macro_next_riskoff',  
'hmm_macro_entropy']
```

5. Model selection

Each instrument runs a horse race across penalised logistic regression, a random forest, gradient boosting and a small multi-task network, scored by combinatorial purged cross-validation (six groups, two held out, fifteen paths) with per-instrument purging and embargo. Feature variants (full, pruned, reduced) are compared, and the simplest variant within one cross-path standard error of the best is locked **before** the out-of-sample window is touched. The cross-validation runs on the training partition only; the sealed test (after 2021-10-20) is scored once, with the locked choice fixed.

Random forests win across the metals, and the precious names are pooled where their individual event counts are thin. The cautionary result is gold: a reduced feature set lifts its cross-validated AUC above 0.56, yet the same model drops below 0.5 out of sample. That pattern — strong in development, chance in the hold-out — is the signature of fitting to the top few importance clusters rather than to anything that generalises. The locked picks below are honest about which metals clear their confidence bound and which do not.

```
In [6]: mc = OUT / "model_comparison" / CLASS  
sel = load(OUT / "model_comparison/selection_table.csv")  
locked = load(mc / "locked_picks.csv")  
cpcv = load(mc / "cpcv_results.csv")  
oos = load(mc / "oos_results.csv")  
  
if not sel.empty:  
    # NB: selection_table's own 'n_events' is a derived training-sample  
    # count, not the  
    # triple-barrier event count reported in section 3, so it is omitted  
    # here to avoid a  
    # misleading clash.  
    keep = [c for c in ["instrument", "best_model", "best_auc", "lower_ci",  
"signal"]  
            if c in sel.columns]  
    sc = sel[sel["instrument"].isin(INSTS)]  
    [keep].set_index("instrument").reindex(INSTS).reset_index()  
    table(sc, caption=f"{CLASS.title()} – champion model and signal verdict  
(selection_table.csv)")  
  
if not locked.empty:  
    table(locked, caption="Locked feature variant per instrument (1SE rule,  
train-only)")  
  
for inst in INSTS:
```

```

display(Markdown(f"**{NAMES[inst]} - `{inst}` cross-validated
variants**"))
show(mc / f"{inst}_cpcv_chart.png", width=760)

show(mc / "oos_summary_chart.png", width=820)

# Out-of-sample AUC for the locked variants only.
if not oos.empty and not locked.empty:
    lk = dict(zip(locked["inst"], locked["locked_variant"]))
    mask = oos.apply(lambda r: lk.get(r["inst"]) == r["variant"], axis=1)
    keep = [c for c in ["inst", "variant", "auc", "auc_ci_lo", "auc_ci_hi",
"n_test"] if c in oos.columns]
    table(oos[mask][keep], caption="Out-of-sample AUC - locked variants
only")

```

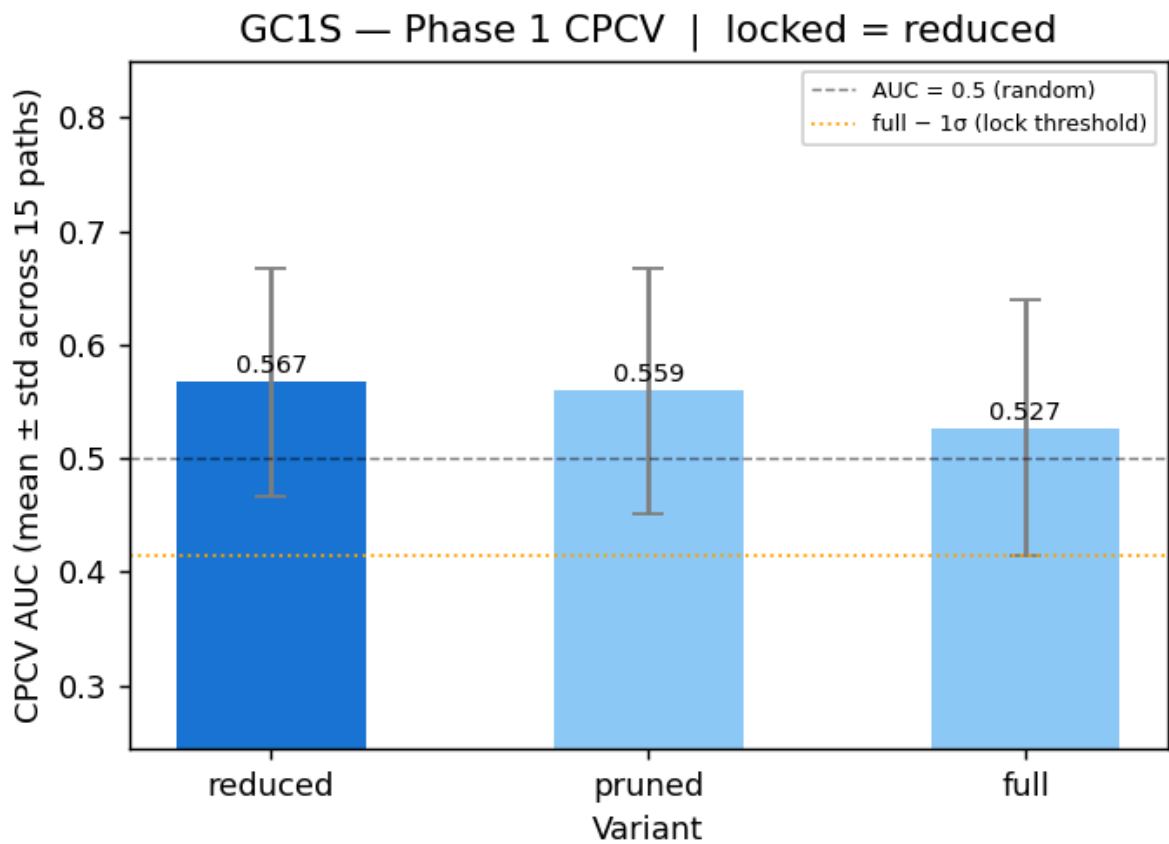
Metals — champion model and signal verdict
(selection_table.csv)

instrument	best_model	best_auc	lower_ci	signal
gc1s	rf	0.4866	0.3734	False
si1s	rf	0.5400	0.4762	False
hg1s	rf	0.5463	0.5058	True
pl1s	rf	0.5697	0.5396	True

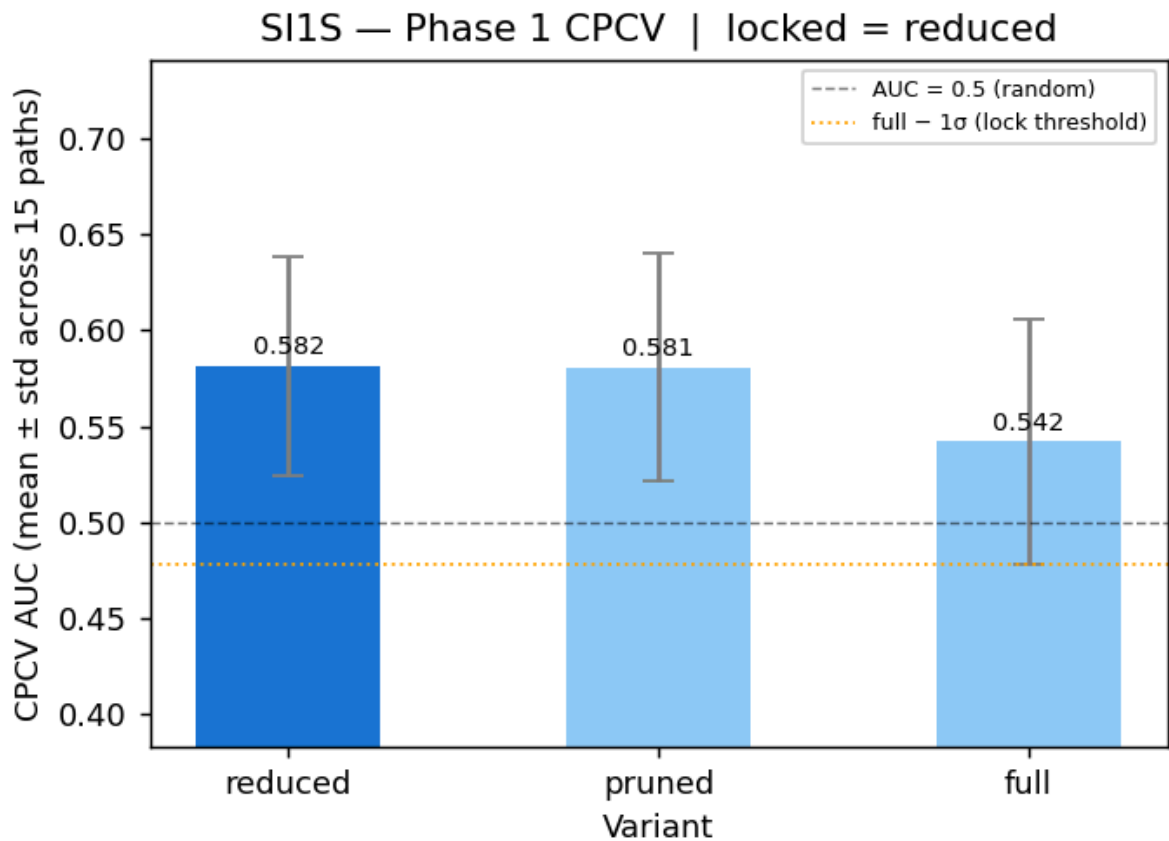
Locked feature variant
per instrument (1SE rule,
train-only)

inst	locked_variant
gc1s	reduced
si1s	reduced
pl1s	reduced
hg1s	reduced

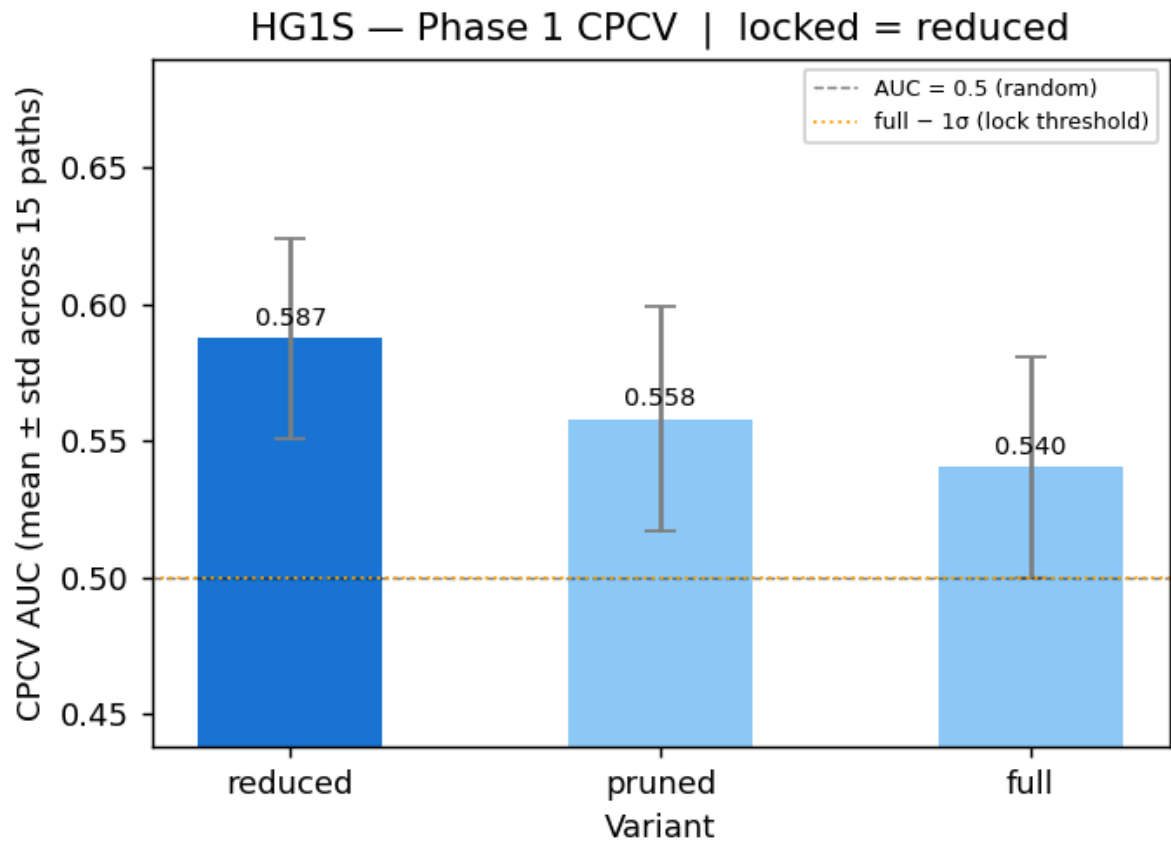
Gold (GC) — **gc1s** cross-validated variants



Silver (SI) — `si1s` cross-validated variants

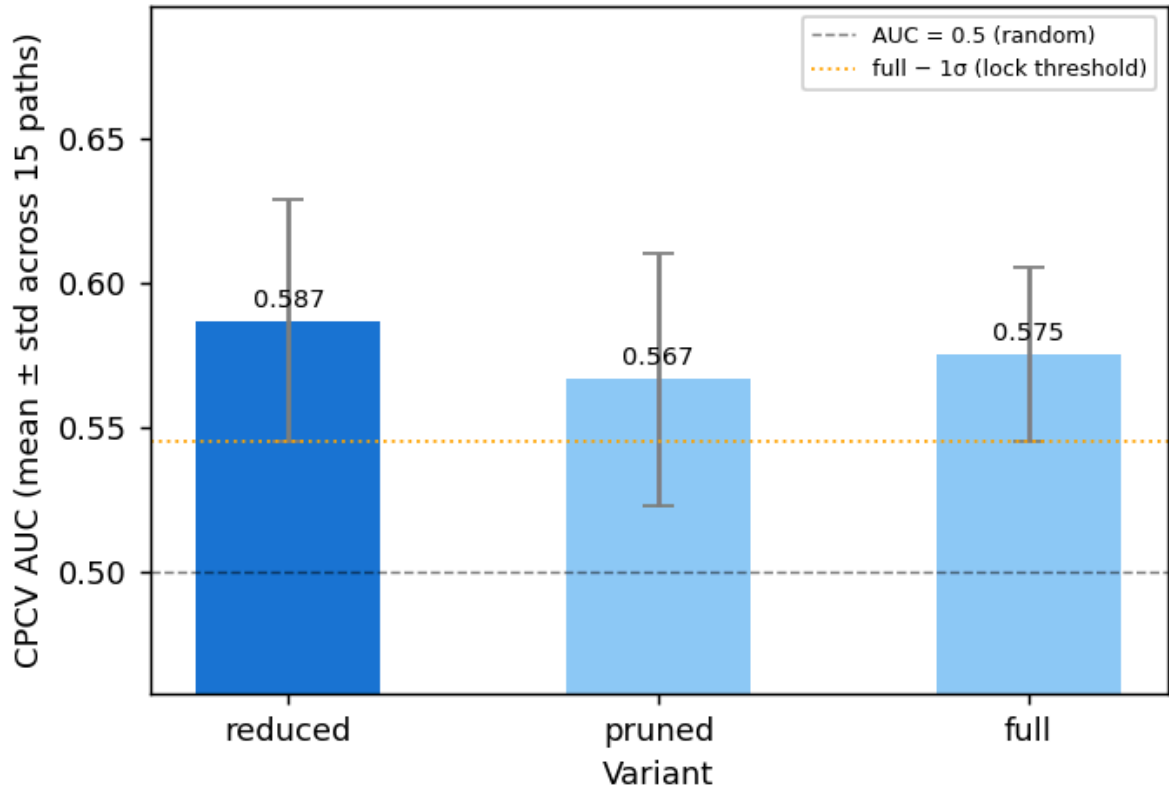


Copper (HG) — hg1s cross-validated variants

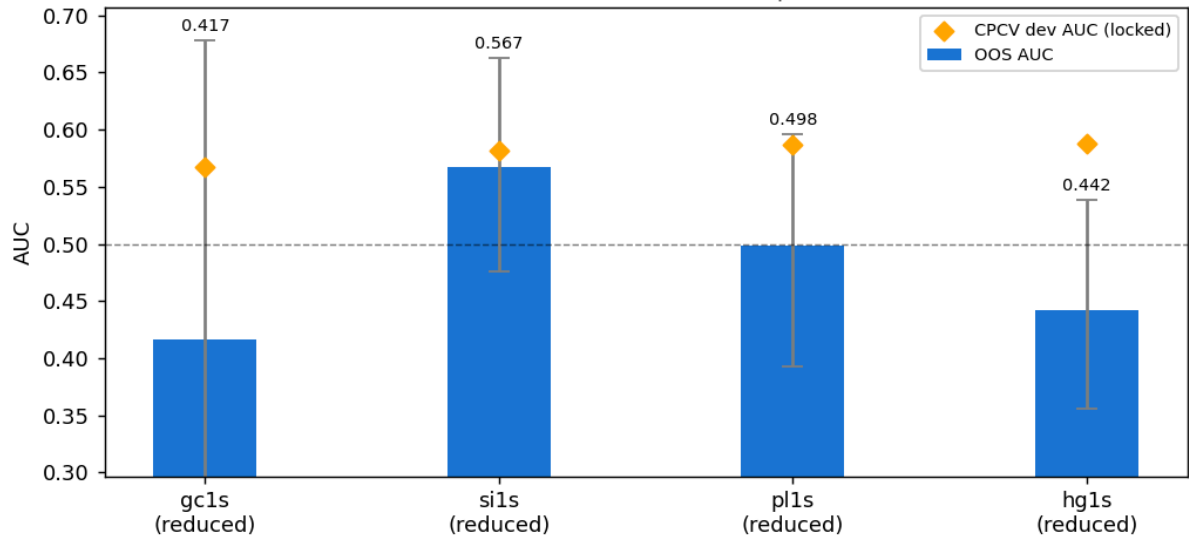


Platinum (PL) — pl1s cross-validated variants

PL1S — Phase 1 CPCV | locked = reduced



Metals — Phase 2 OOS (locked variants) | ♦ = CPCV dev AUC



Out-of-sample AUC — locked variants only

inst	variant	auc	auc_ci_lo	auc_ci_hi	n_test
gc1s	reduced	0.4167	0.1665	0.6786	23
si1s	reduced	0.5668	0.4759	0.6628	144
pl1s	reduced	0.4984	0.3933	0.5958	148
hg1s	reduced	0.4421	0.3559	0.5389	164

6. Feature importance

Importance is read at the cluster level rather than per raw feature, because the predictors are correlated in blocks and per-feature scores would split credit arbitrarily across substitutes. Features are clustered on a rank-correlation distance; the dendrogram shows the block structure, mean-decrease-accuracy gives each cluster's drop, and a global SHAP or coefficient ranking resolves the within-cluster detail. Logistic champions are read through their coefficients, tree and network champions through SHAP.

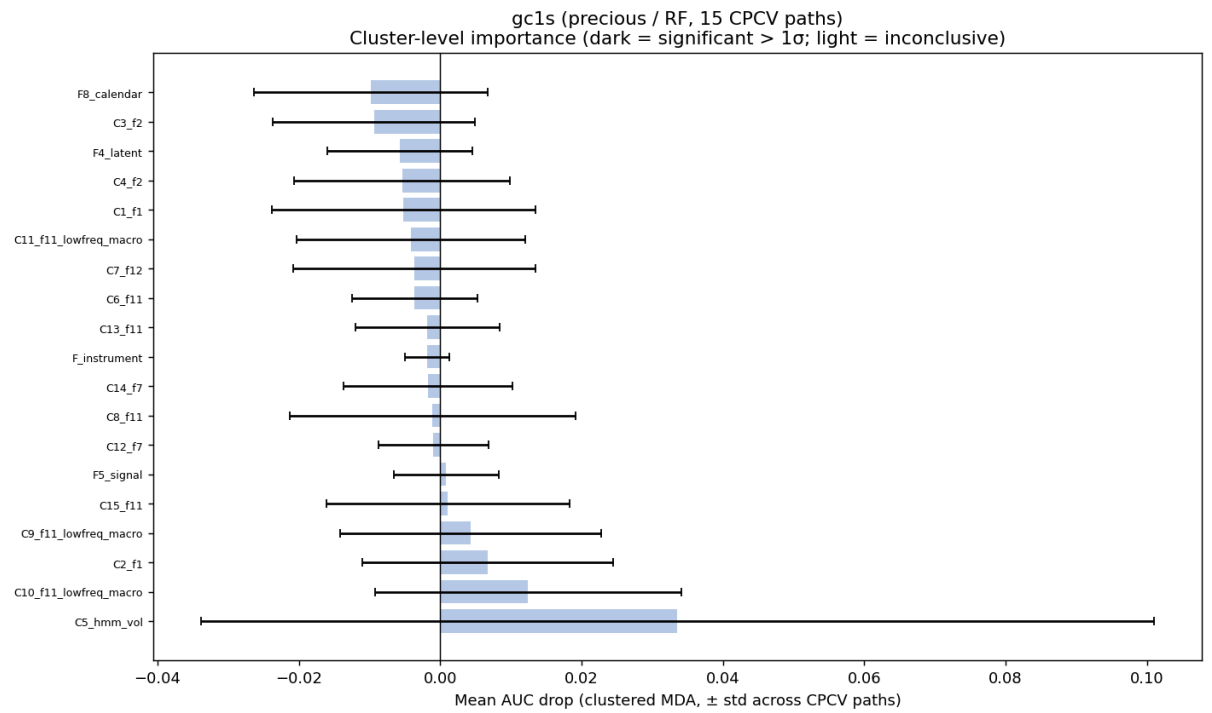
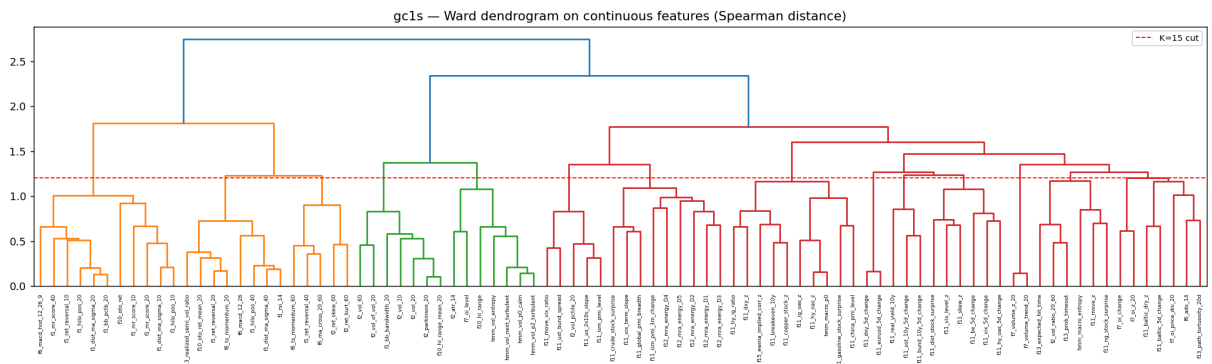
The clustered importance plots for metals are worth reading against the model caveat. Where a name's apparent edge rests on two or three significant clusters, the reduced-feature model that exploits them is exactly the one that fails to generalise. Silver and gold illustrate this; copper and platinum present a steadier, if modest, picture. The dendrograms make the block structure of the metals feature set visible, which is what motivates clustering the importances in the first place.

```
In [7]: imp = OUT / "importance"
sel = load(OUT / "model_comparison/selection_table.csv")
model_of = dict(zip(sel.get("instrument", []), sel.get("best_model", [])))
if not sel.empty else {}

for inst in INSTS:
    d = imp / inst
    display(Markdown(f"### {NAMES[inst]} - `{inst}`"))
    cm = load(d / "champion_meta.csv")
    if not cm.empty:
        table(cm)
    show(d / "dendrogram.png", width=900)
    show(d / "clustered_mda_chart.png", width=820)
    mda = load(d / "clustered_mda_full.csv")
    if not mda.empty:
        table(mda.head(8), caption="Top clusters by mean decrease in accuracy")
    coef_png = d / "global_coef_chart.png"
    if str(model_of.get(inst, "")) == "logistic" and coef_png.exists():
        show(coef_png, width=820)
        table(load(d / "global_coef_summary.csv").head(15), caption="Top logistic coefficients")
    else:
        show(d / "global_shap_chart.png", width=820)
        table(load(d / "global_shap_summary.csv").head(15), caption="Top features by |SHAP|")
```

Gold (GC) — gc1s

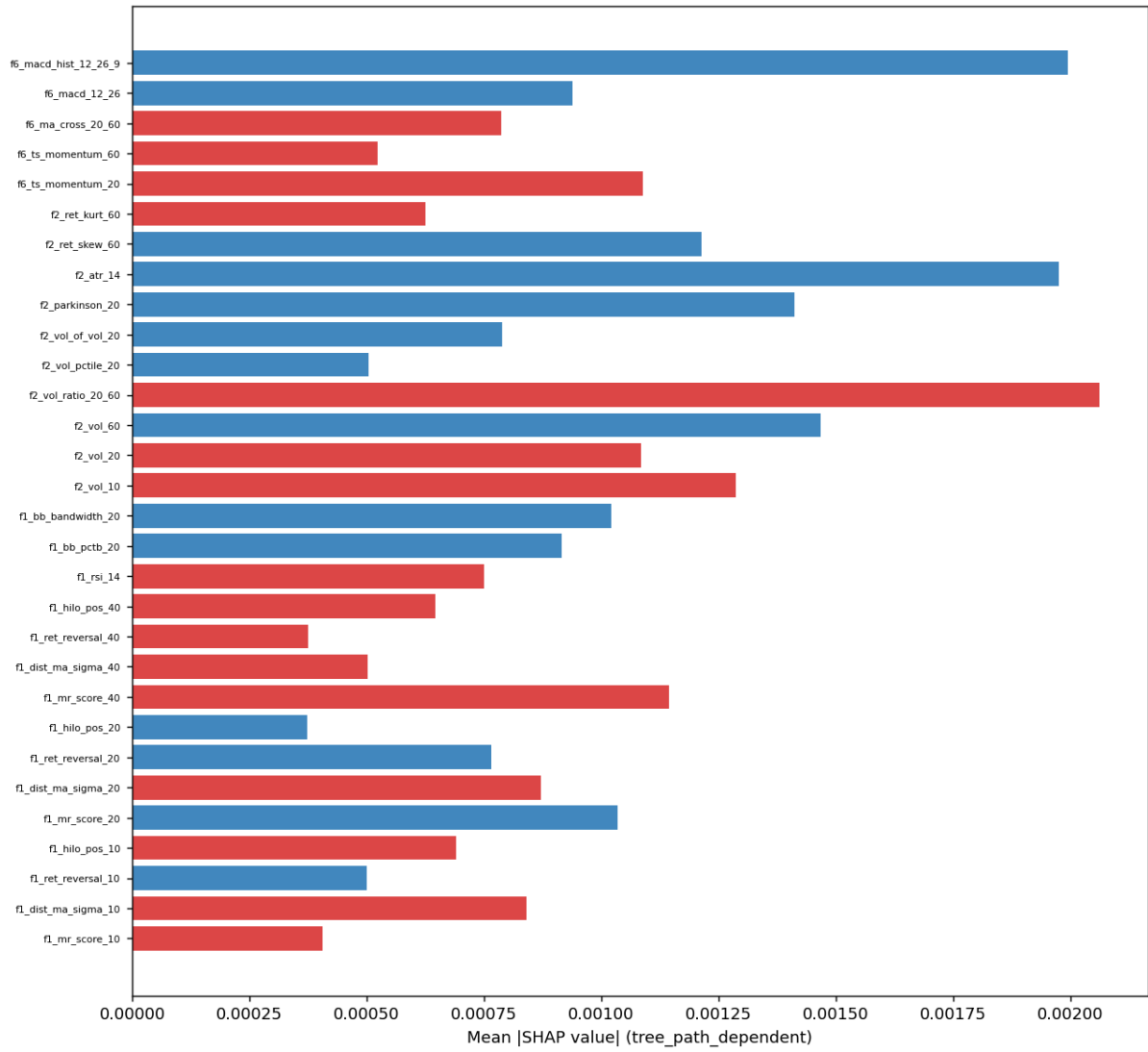
instrument	asset_class	group	family	model_type	auc_mean	auc_std	lower_ci	si
gc1s	metals	precious	tree	rf	0.4866	0.1132	0.3734	F



Top clusters by mean decrease in accuracy

cluster	mean_drop	std_drop	significant
C5_hmm_vol	0.0336	0.0674	False
C10_f11_lowfreq_macro	0.0124	0.0217	False
C2_f1	0.0067	0.0178	False
C9_f11_lowfreq_macro	0.0043	0.0184	False
C15_f11	0.0011	0.0172	False
F5_signal	0.0008	0.0074	False
C12_f7	-0.0010	0.0078	False
C8_f11	-0.0011	0.0202	False

gc1s — Global per-feature SHAP (top 30)
red=positive signal, blue=negative

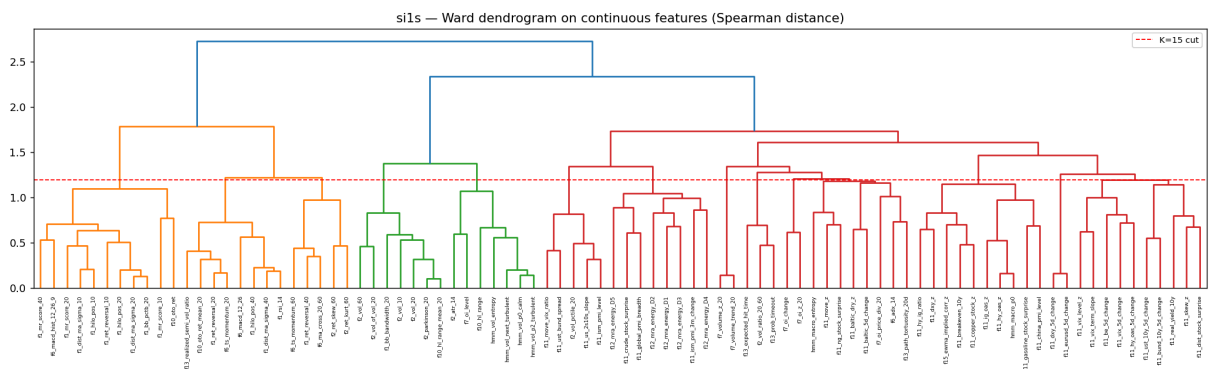


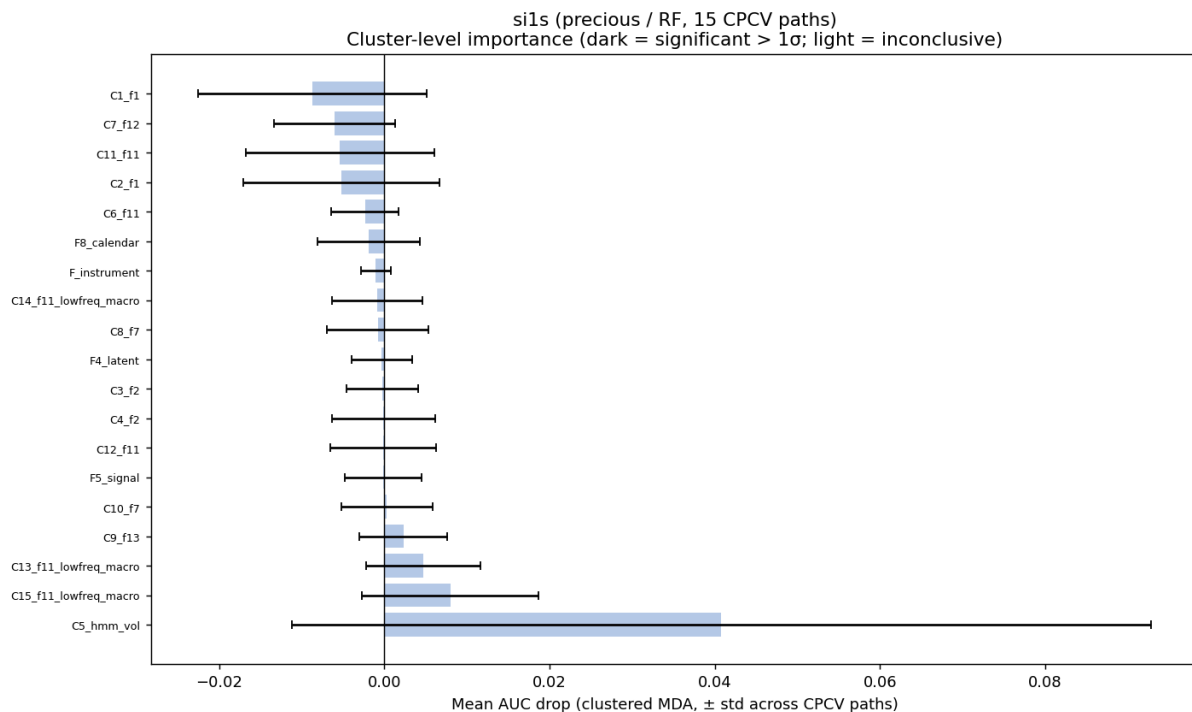
Top features by |SHAP|

feature	shap_magnitude	shap_signed	mdi
hmm_vol_p0_calm	0.0085	-0.0007	0.0432
hmm_vol_p2_turbulent	0.0079	-0.0017	0.0427
hmm_vol_next_turbulent	0.0079	-0.0015	0.0463
f11_ust_10y_5d_change	0.0037	0.0001	0.0219
hmm_vol_entropy	0.0030	-0.0006	0.0278
f6_adx_14	0.0029	-0.0018	0.0307
f11_eurusd_5d_change	0.0026	0.0000	0.0206
f7_oi_z_20	0.0026	-0.0005	0.0172
f4_pc2	0.0025	-0.0010	0.0187
f11_copper_stock_z	0.0023	0.0012	0.0109
f4_pc1	0.0022	-0.0001	0.0164
f10_hl_range	0.0022	-0.0015	0.0104
f11_dist_stock_surprise	0.0021	-0.0004	0.0122
f2_vol_ratio_20_60	0.0021	0.0001	0.0111
f6_macd_hist_12_26_9	0.0020	-0.0002	0.0226

Silver (SI) — si1s

instrument	asset_class	group	family	model_type	auc_mean	auc_std	lower_ci	si
si1s	metals	precious	tree	rf	0.5400	0.0638	0.4762	F

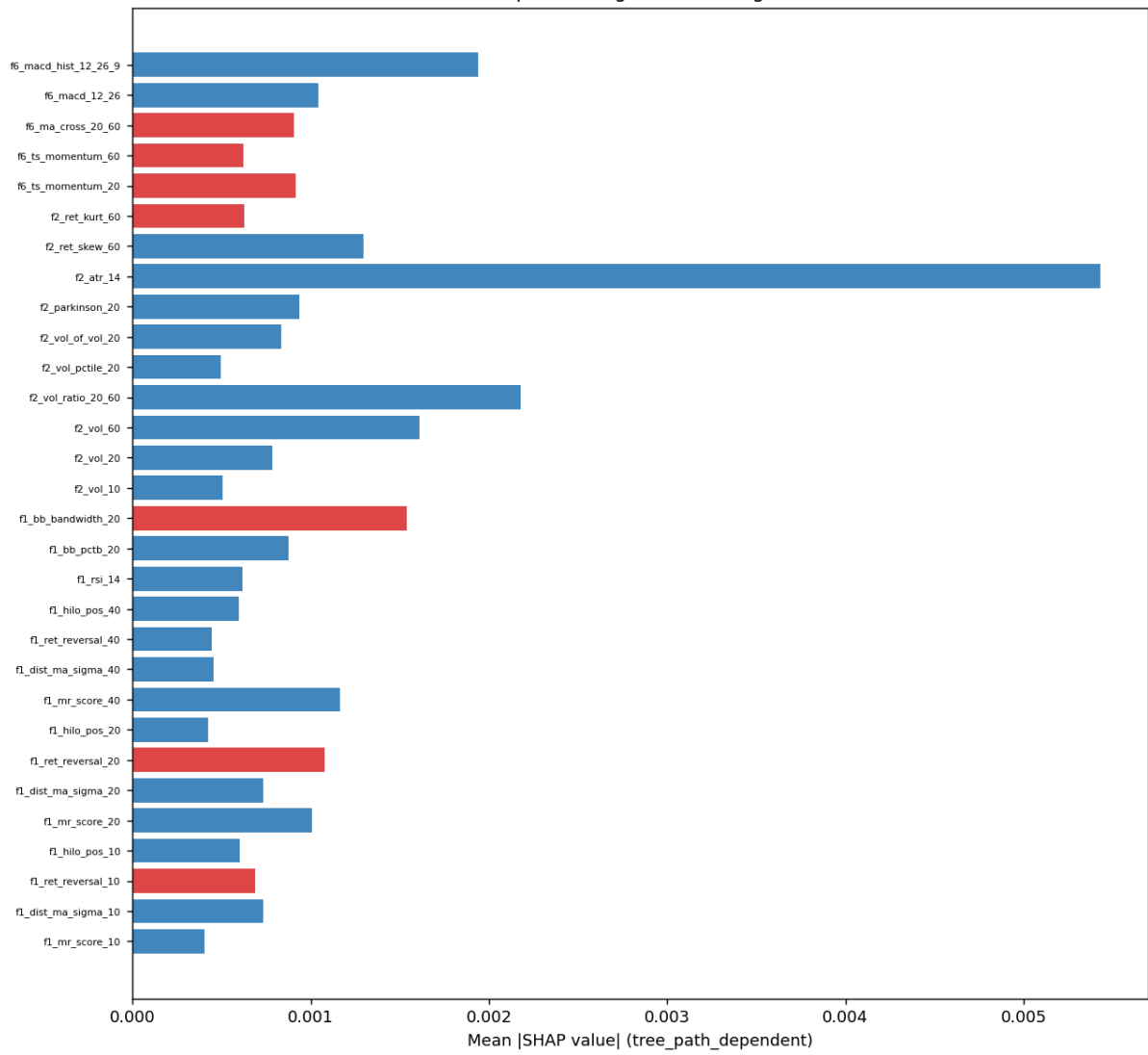




Top clusters by mean decrease in accuracy

cluster	mean_drop	std_drop	significant
C5_hmm_vol	0.0408	0.0520	False
C15_f11_lowfreq_macro	0.0080	0.0107	False
C13_f11_lowfreq_macro	0.0047	0.0069	False
C9_f13	0.0023	0.0053	False
C10_f7	0.0003	0.0055	False
F5_signal	-0.0001	0.0046	False
C12_f11	-0.0001	0.0064	False
C4_f2	-0.0001	0.0062	False

si1s — Global per-feature SHAP (top 30)
red=positive signal, blue=negative

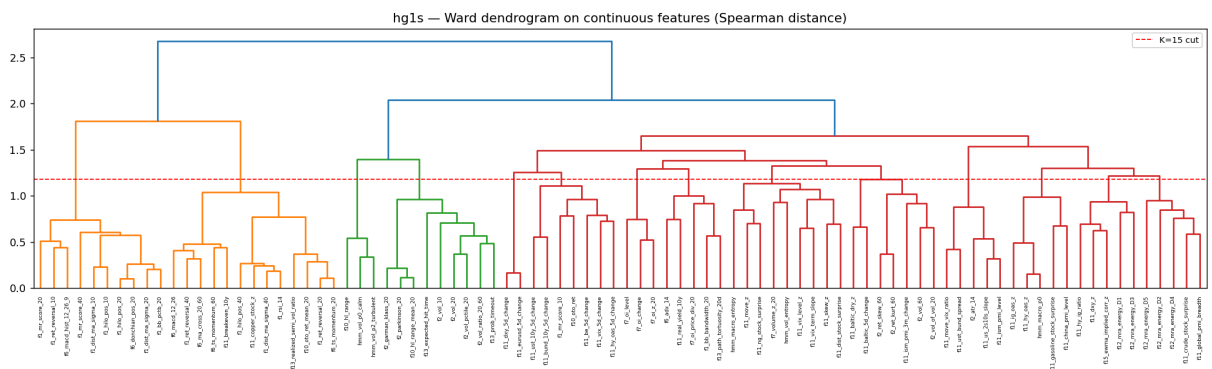


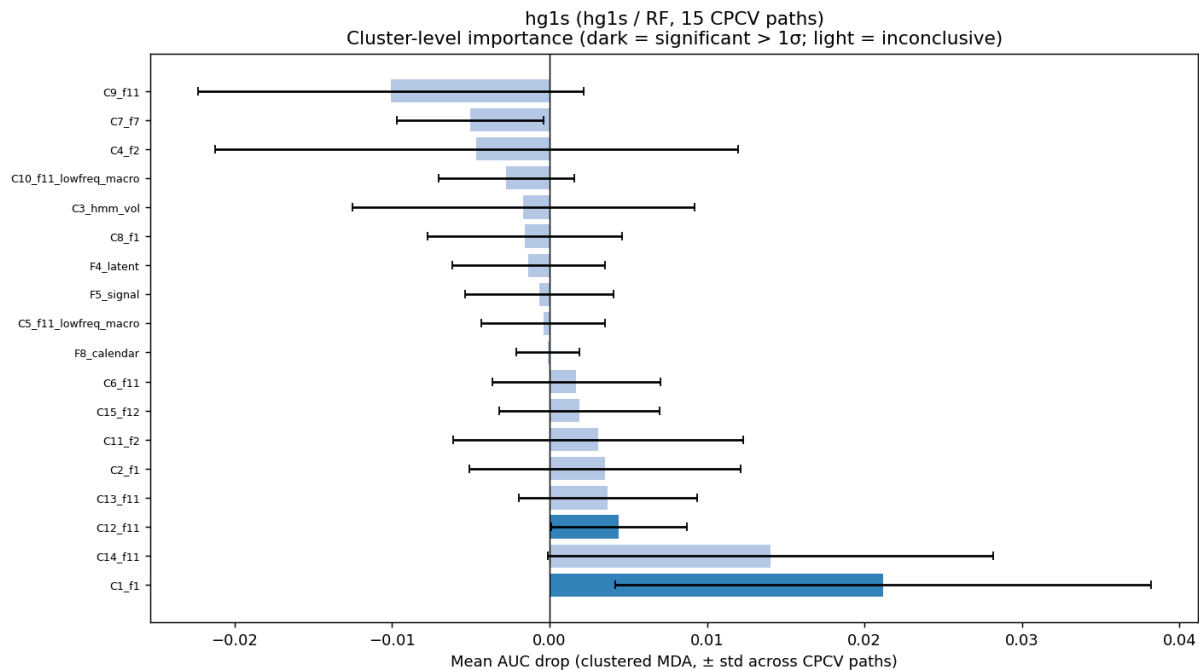
Top features by |SHAP|

feature	shap_magnitude	shap_signed	mdi
hmm_vol_p0_calm	0.0101	-0.0040	0.0432
hmm_vol_p2_turbulent	0.0091	-0.0036	0.0427
hmm_vol_next_turbulent	0.0088	-0.0035	0.0463
f2_atr_14	0.0054	-0.0054	0.0301
f11_ust_10y_5d_change	0.0038	-0.0005	0.0219
hmm_vol_entropy	0.0031	-0.0009	0.0278
f6_adx_14	0.0031	-0.0017	0.0307
f11_eurusd_5d_change	0.0025	-0.0002	0.0206
f4_pc2	0.0025	-0.0005	0.0187
f7_oi_z_20	0.0025	-0.0005	0.0172
f11_copper_stock_z	0.0023	0.0013	0.0109
f2_vol_ratio_20_60	0.0022	-0.0002	0.0111
f11_bund_10y_5d_change	0.0021	-0.0002	0.0168
f7_oi_level	0.0021	-0.0020	0.0163
inst_si1s	0.0019	-0.0019	0.0061

Copper (HG) — hg1s

instrument	asset_class	group	family	model_type	auc_mean	auc_std	lower_ci	sign
hg1s	metals	hg1s	tree	rf	0.5463	0.0405	0.5058	Tri

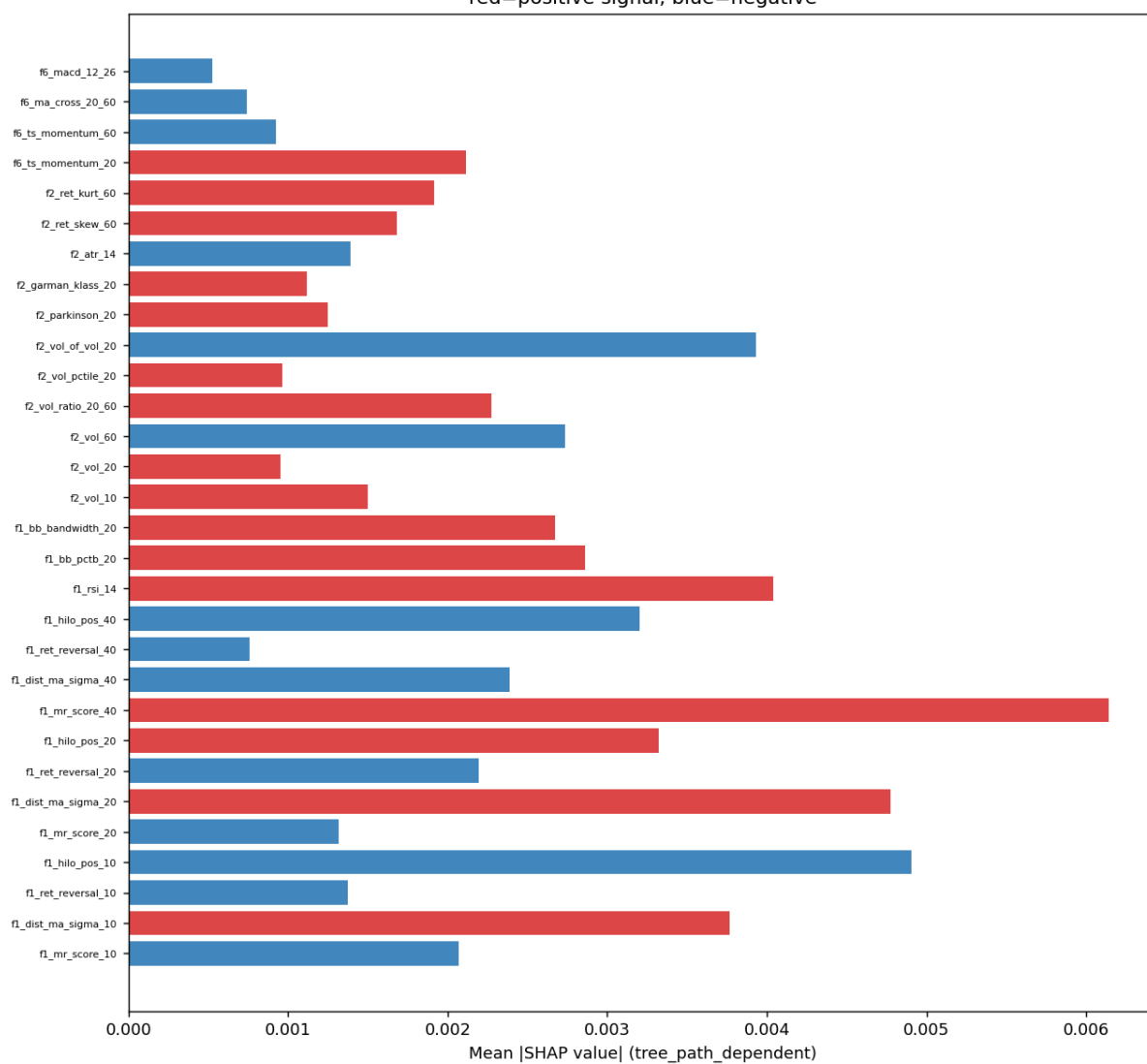




Top clusters by mean decrease in accuracy

cluster	mean_drop	std_drop	significant
C1_f1	0.0212	0.0170	True
C14_f11	0.0140	0.0141	False
C12_f11	0.0044	0.0043	True
C13_f11	0.0037	0.0057	False
C2_f1	0.0035	0.0086	False
C11_f2	0.0031	0.0092	False
C15_f12	0.0019	0.0051	False
C6_f11	0.0017	0.0054	False

hg1s — Global per-feature SHAP (top 30)
red=positive signal, blue=negative

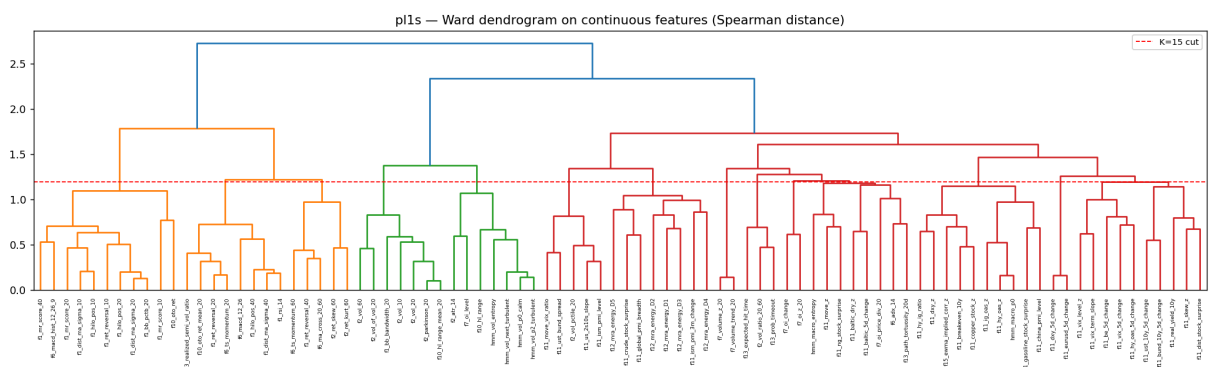


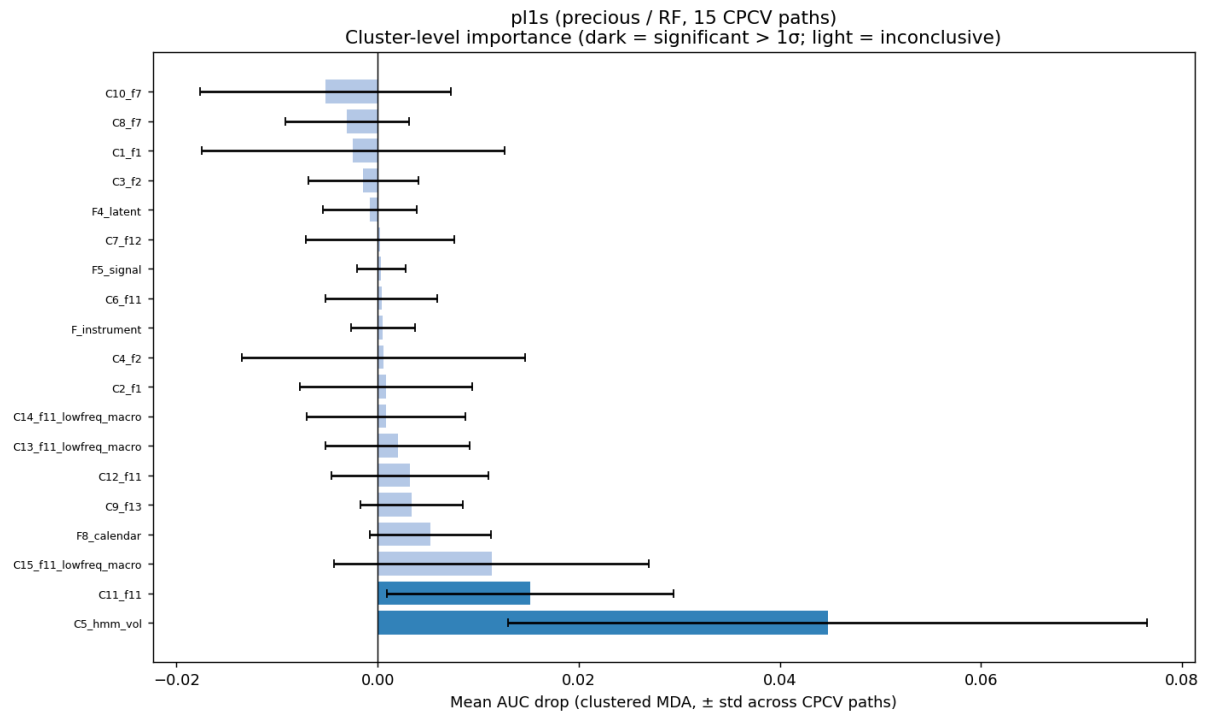
Top features by |SHAP|

feature	shap_magnitude	shap_signed	mdi
f1_mr_score_40	0.0061	0.0004	0.0258
f12_mra_energy_D1	0.0055	-0.0014	0.0314
f1_hilo_pos_10	0.0049	-0.0003	0.0307
f1_dist_ma_sigma_20	0.0048	0.0016	0.0259
f12_mra_energy_D4	0.0046	0.0006	0.0225
f1_rsi_14	0.0040	0.0002	0.0215
f2_vol_of_vol_20	0.0039	-0.0014	0.0171
f11_skew_z	0.0039	0.0008	0.0201
f1_dist_ma_sigma_10	0.0038	0.0002	0.0245
f11_gasoline_stock_surprise	0.0037	-0.0000	0.0195
f11_dxy_z	0.0034	-0.0003	0.0161
f1_hilo_pos_20	0.0033	0.0001	0.0203
f1_hilo_pos_40	0.0032	-0.0001	0.0181
f11_dist_stock_surprise	0.0031	0.0002	0.0142
f11_copper_stock_z	0.0031	-0.0004	0.0145

Platinum (PL) — pl1s

instrument	asset_class	group	family	model_type	auc_mean	auc_std	lower_ci	si
pl1s	metals	precious	tree	rf	0.5697	0.0301	0.5396	

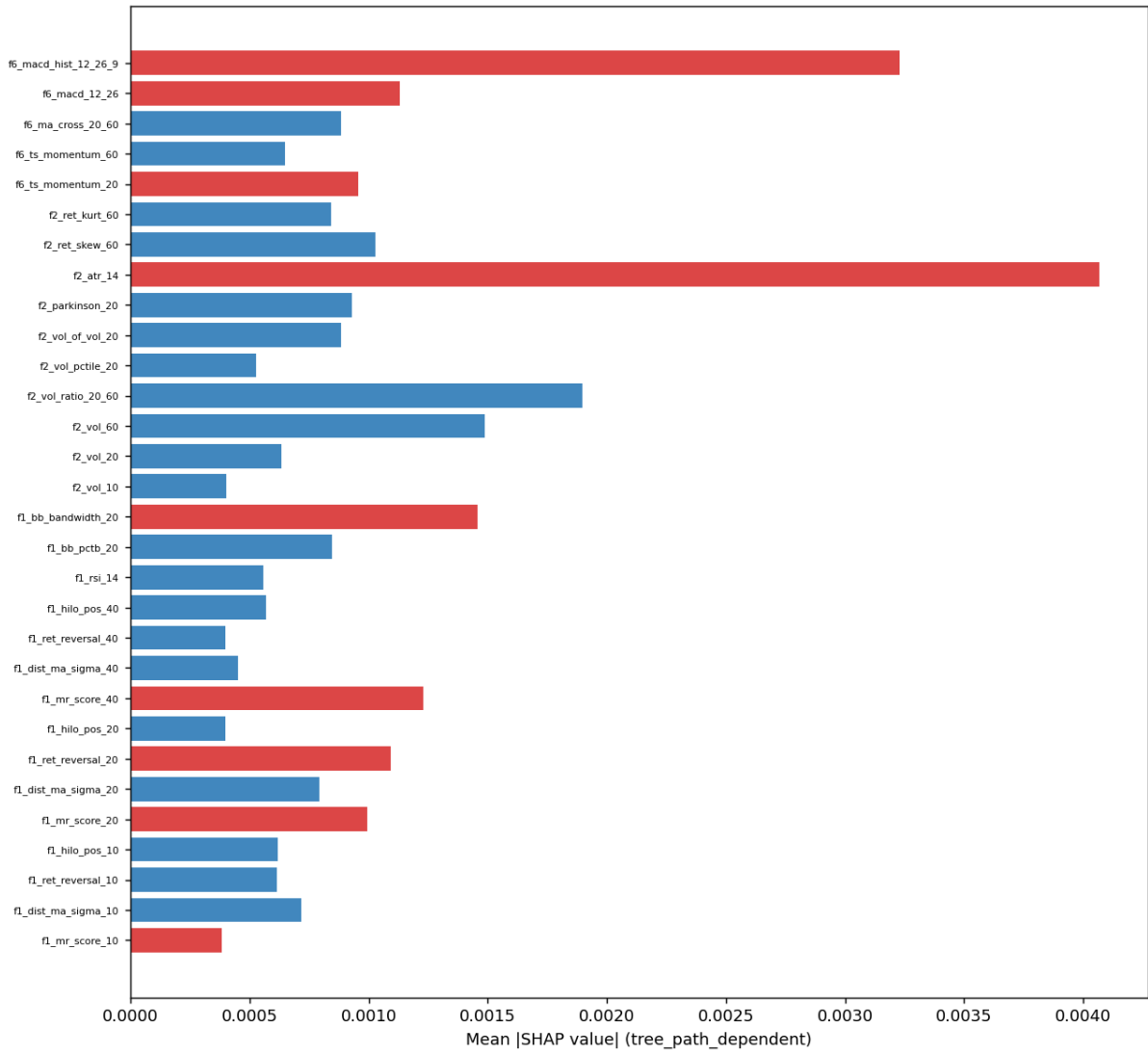




Top clusters by mean decrease in accuracy

cluster	mean_drop	std_drop	significant
C5_hmm_vol	0.0447	0.0318	True
C11_f11	0.0152	0.0142	True
C15_f11_lowfreq_macro	0.0113	0.0157	False
F8_calendar	0.0053	0.0060	False
C9_f13	0.0034	0.0051	False
C12_f11	0.0032	0.0078	False
C13_f11_lowfreq_macro	0.0020	0.0072	False
C14_f11_lowfreq_macro	0.0009	0.0079	False

pl1s — Global per-feature SHAP (top 30)
red=positive signal, blue=negative



Top features by |SHAP|

feature	shap_magnitude	shap_signed	mdi
f6_adx_14	0.0072	0.0038	0.0307
hmm_vol_p0_calm	0.0069	0.0039	0.0432
hmm_vol_next_turbulent	0.0066	0.0039	0.0463
hmm_vol_p2_turbulent	0.0063	0.0038	0.0427
f2_atr_14	0.0041	0.0039	0.0301
f11_ust_10y_5d_change	0.0038	-0.0005	0.0219
f6_macd_hist_12_26_9	0.0032	0.0013	0.0226
f7_oi_z_20	0.0025	-0.0001	0.0172
hmm_vol_entropy	0.0024	0.0007	0.0278
f11_eurusd_5d_change	0.0024	-0.0001	0.0206
f11_copper_stock_z	0.0022	0.0013	0.0109
f11_dist_stock_surprise	0.0022	-0.0003	0.0122
f11_bund_10y_5d_change	0.0022	-0.0002	0.0168
f4_pc1	0.0020	-0.0004	0.0164
f4_pc2	0.0020	0.0009	0.0187

7. Strategy — pooled portfolio and per-class attribution

There is a single backtest, not one per class. The live strategy is one volatility-targeted, equally-weighted book over all eleven contracts: an EWMA(60) volatility estimate scales each position to a 10% annualised target, leverage is capped, returns lag the weights by a day, and Grinold-Kahn costs (a 2bp half-spread plus 10bp of turnover) are charged. The conventions are recorded in `results/strategy_eval/eval_summary.md`. Method A follows every signal; Method B trades only when the calibrated metamodel probability exceeds one half.

The headline table and cumulative-return figures below are the **pooled** result. The per-class table that follows is an **attribution** — this class's slice of the shared book — and not a standalone per-class Sharpe; with one pooled portfolio the instruments cannot be isolated into independent backtests.

```
In [8]: se = RESULTS / "strategy_eval"
table(load(se / "eval_summary.csv"), caption="Pooled portfolio – 00S
headline (eval_summary.csv)")
show(se / "cumulative_returns.png", width=860)
show(se / "per_asset_cumulative_returns.png", width=860)
```

```

show(se / "per_instrument_contribution.png", width=860)

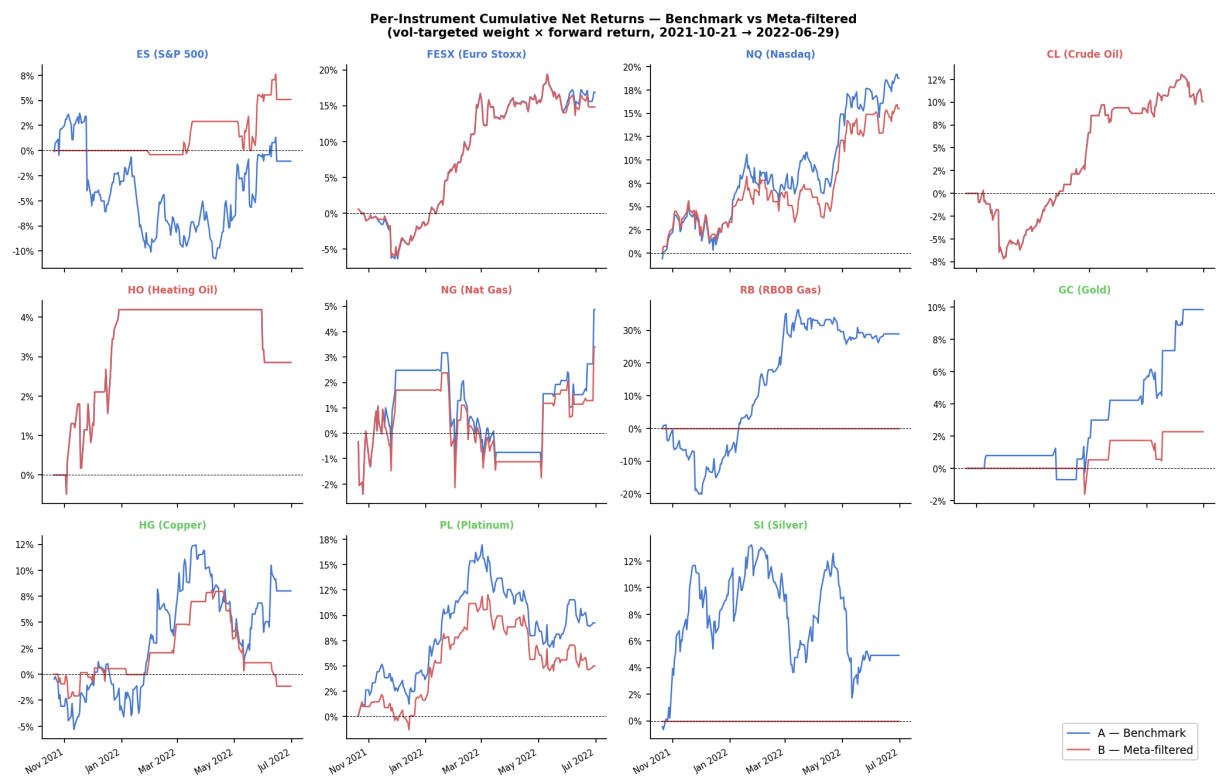
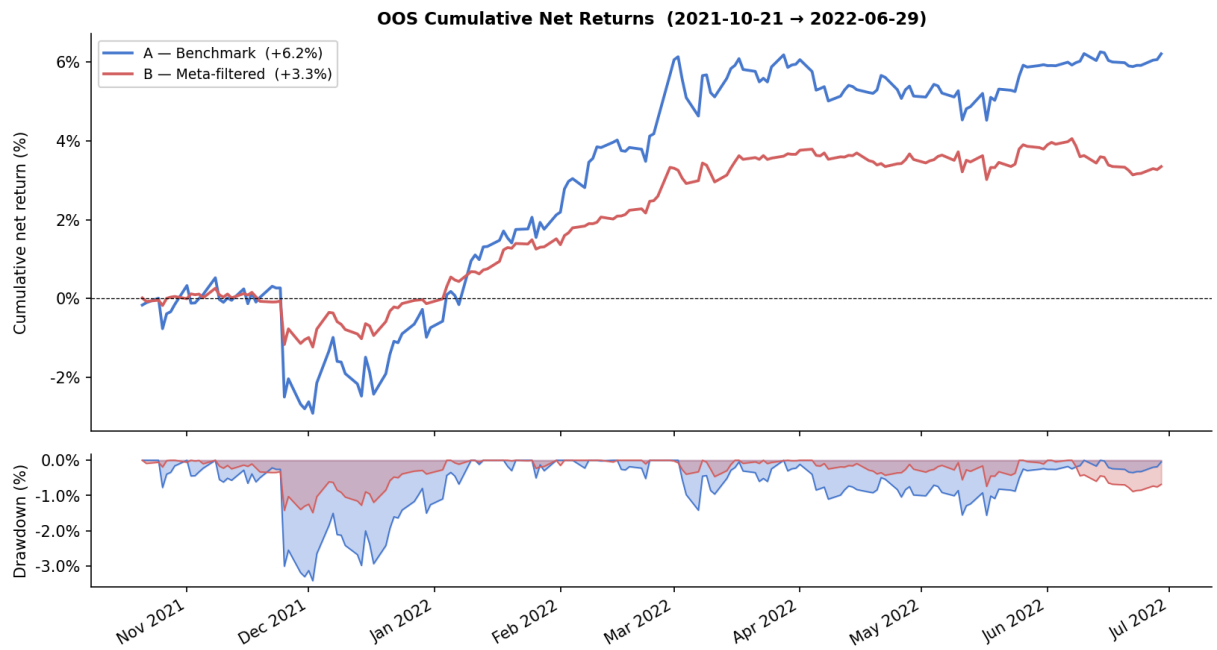
# --- Per-class attribution within the pooled 1/K book (with numeric
guards) ---
preds = load(OUT / "metamodel_predictions.csv")
assert not preds.empty, "metamodel_predictions.csv missing"
assert preds["calibrated_proba"].between(0, 1).all(), "calibrated_proba
outside [0, 1]"
assert set(INSTS) <= set(preds["instrument"].unique()), "class instruments
absent from predictions"

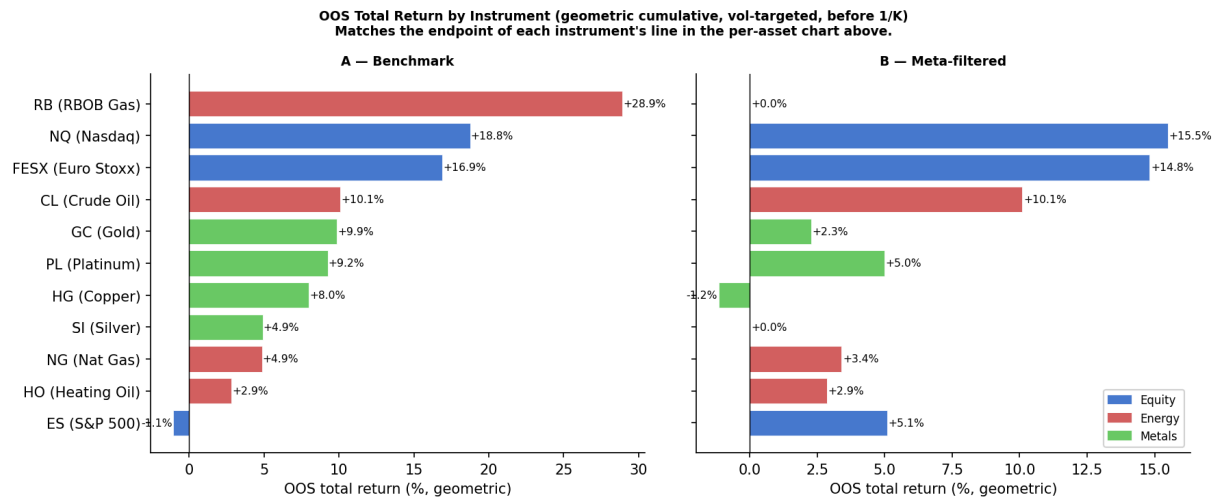
cls = preds[preds["instrument"].isin(INSTS)].copy()
cls["taken"] = cls["calibrated_proba"] > 0.5
attr = (cls.groupby("instrument")
        .apply(lambda g: pd.Series({
            "n_events": len(g),
            "events_taken": int(g["taken"].sum()),
            "mean_calib_proba": g["calibrated_proba"].mean(),
            "label1_share": g["bin"].mean(),
            "hit_rate_taken": g.loc[g["taken"], "bin"].mean() if
g["taken"].any() else float("nan"),
        }), include_groups=False)
        .reindex(INSTS).reset_index())
assert (attr["events_taken"] <= attr["n_events"]).all(), "events_taken
exceeds n_events"
table(attr, caption=f"{CLASS.title()} – attribution within the pooled book
(metamodel_predictions.csv)")

```

Pooled portfolio — OOS headline (eval_summary.csv)

	Metric	A Benchmark	B all_or_nothing	B SOPS
	Ann return	+8.62%	+4.65%	+4.65%
	Ann vol	6.10%	2.88%	2.88%
	Sharpe	1.412	1.614	1.614
	t-stat	1.19	1.36	1.36
	SR 95% CI low	-1.559	-1.286	-1.286
	SR 95% CI high	3.750	3.821	3.821
	PSR(SR*=0)	0.861	0.899	0.899
	Max DD	-3.42%	-1.49%	-1.49%
	Total cost (bps)	416.1	188.4	188.4
	Annual turnover	48.28	21.86	21.86
	Breakeven ½-spread	17.8 bps	21.3 bps	21.3 bps
	Events taken	100%	54%	54%





Metals — attribution within the pooled book (metamodel_predictions.csv)

instrument	n_events	events_taken	mean_calib_proba	label1_share	hit_rate_taken
gc1s	23.0000	7.0000	0.4458	0.6522	0.5714
si1s	144.0000	0.0000	0.3570	0.4722	nan
hg1s	164.0000	50.0000	0.4586	0.3841	0.3200
pl1s	148.0000	119.0000	0.5110	0.5541	0.5462

8. Discussion

Metals close the study on a deliberately sober note. The signal is the cleanest in the universe, yet the metamodel edge is the weakest: gold is effectively random out of sample, silver is marginal, and only copper and platinum offer a steady if small contribution. The lesson is methodological as much as empirical — a high cross-validated score on a reduced feature set is not evidence of an exploitable edge, and the hold-out is the only arbiter that matters.

References

- López de Prado, M. (2018) *Advances in Financial Machine Learning*. Hoboken, NJ, Wiley. [Triple-barrier labelling ch. 3; average uniqueness ch. 4; purged and combinatorial cross-validation ch. 7.]
- Project methodology notes (this repository): `reports/harry/00-context.md`, `01-signal-direction.md`, `02-labels.md`, `03-features.md`, `03-6-macro-features.md`, and `reports/missing-data-report.md`.
- Strategy conventions: `results/strategy_eval/eval_summary.md` (StrategyWeights lecture, slides 38–43).
- Course materials: *Systematic Trading Strategies with Machine Learning* (T3.03), lectures L1–L8.